

JavaScript Collaborative Group Activities

Submission Date: Saturday, 14th August, 2026

A Beginner-Friendly Workshop Workbook

This workbook contains 10 collaborative group activities designed to help beginner JavaScript students move from *memorizing* concepts to *understanding, explaining, and applying* them. Each activity should take approximately 30–45 minutes.

Activity 1: The Digital Memory Box

Topics Covered

- Introduction to JavaScript
- `console.log()`
- `alert()`/`prompt()`
- `confirm()`
- Variables
- Data Types

Learning Objective

By the end of this activity, students should be able to explain why programs need to store information, choose sensible variable names, and identify the basic data types JavaScript uses to represent that information.

Part 1: Group Discussion

1. Imagine you're building a program that needs to remember a user's name so it can greet them later. How would a variable help the program do this?
2. What's the difference between `console.log()`, `alert()`, `prompt()`, and `confirm()`? When might a programmer choose one over another?
3. If you declared a variable with `let` instead of `const`, what would that let you do that you couldn't do otherwise? Why might a programmer choose `const` on purpose, even though `let` seems more flexible?
4. JavaScript has several basic data types (numbers, strings, booleans, etc.). Why do you think a programming language needs different types instead of just one type for everything?
5. If two people on your team pick different variable names for the same piece of information (e.g., `userName` vs `name1`), what problems could that cause later?

Part 2: Predict the Output

Snippet A

```
let age = 25;  
console.log(age);  
  
age = "twenty-five";  
console.log(age);
```

What will this code output?

Snippet B

```
const isRaining = true;  
console.log("Is it raining? " + isRaining);
```

What will this code output?

Snippet C

```
let favoriteNumber;  
console.log(favoriteNumber);
```

What will this code output?

Part 3: Debugging Challenge

```
let userName = "Amara"  
console.log(username);  
  
Const favoriteColor = "blue";  
console.log(favoriteColor);
```

Discuss as a group: What's wrong with this code? Once you've identified the issues, fix them and explain how the corrected version works.

Part 4: Collaborative Technical Challenge

Scenario: Your group needs to build a simple "Welcome Program" that greets a new user by name and tells them what year they were born, based on their age.

Step 1 — Understand the Problem: In your own words, explain what this program needs to do.

Step 2 — Identify Inputs: What information does the program need from the user?

Step 3 — Identify Processing: What will the program need to calculate or figure out?

Step 4 — Identify Outputs: What should the program display to the user?

Step 5 — Plan the Solution: Before writing any code, write out the steps in plain English.

Step 6 — Write the Code: Use `prompt()` to collect the user's name and age, store them in variables, and use `console.log()` or `alert()` to greet them.

Step 7 — Test the Program: Try these test cases:

- Name: "Sam", Age: 20
- Name: "Bola", Age: 45

Step 8 — Improve the Solution: As a group, discuss: is your variable naming clear? Could someone else read your code and immediately understand what it does?

Part 5: Extension Challenge

Modify your program so that it also stores the user's favorite hobby in a variable and includes it in the final greeting message. Discuss what data type the hobby should be stored as, and why.

Activity 2: Building Sentences with Code

Topics Covered

- Operators
- String Concatenation
- Template Literals

Learning Objective

Students should be able to combine variables and text into readable output using both concatenation and template literals, and explain the tradeoffs between the two approaches.

Part 1: Group Discussion

1. What's the difference between the `+` operator used for math and the `+` operator used for joining strings? How does JavaScript decide which one to do?
2. Compare this line written two ways: `"Hello, " + name + "! You are " + age + " years old."` vs ``Hello, ${name}! You are ${age} years old.`` Which do you find easier to read, and why?

3. What would happen if you tried to add a number and a string together, like `5 + "5"`? Discuss your predictions before checking.
4. Why do you think template literals use backticks (```) instead of regular quotes?

Part 2: Predict the Output

Snippet A

```
let a = 10;  
let b = "5";  
  
console.log(a + b);  
console.log(a - b);
```

What will this code output?

Snippet B

```
let price = 20;  
let quantity = 3;  
console.log(`Total cost: ${price * quantity}`);
```

What will this code output?

Snippet C

```
let x = 4;  
let y = 2;  
console.log("Result: " + x + y);  
console.log("Result: " + (x + y));
```

What will this code output?

Part 3: Debugging Challenge

```
// 1

let firstName = "Tunde";
let lastName = "Okafor";
let fullName = firstName + " " + lastName;
console.log("Welcome, " + fullname + "!");

// 2

let itemPrice = "15";
let total = itemPrice + 5;
console.log(`Your total is ${total}`);
```

Discuss what's wrong before fixing it.

Part 4: Collaborative Technical Challenge

Scenario: Build a "Receipt Generator" that asks the user for an item name, its price, and how many they're buying, then displays a formatted receipt line.

Step 1 — Understand the Problem: Explain the goal in your own words.

Step 2 — Identify Inputs: What three pieces of information do you need?

Step 3 — Identify Processing: What calculation turns those inputs into a total?

Step 4 — Identify Outputs: What should the final receipt look like?

Step 5 — Plan the Solution: Write the steps in plain English first.

Step 6 — Write the Code: Use variables, a calculation, and a template literal to build the final message.

Step 7 — Test the Program:

- Item: "Notebook", Price: 2.50, Quantity: 4
- Item: "Pen", Price: 0.75, Quantity: 10
- Quantity entered as 0
- Price entered as "free" (non-numeric)
- Quantity entered as 1

Step 8 — Improve the Solution: Could you make the output easier to read by rounding the total or adding a currency symbol? Discuss as a group.

Part 5: Extension Challenge

Add a "discount code" variable to your program. If a discount amount (as a number) is applied, subtract it from the total before displaying the final receipt line.

Activity 3: The Type Detective Agency

Topics Covered

Data Type Conversions, String Methods

Learning Objective

Students should be able to explain why type conversion matters, convert between strings and numbers deliberately, and use common string methods to inspect or transform text.

Part 1: Group Discussion

1. `prompt()` always returns text, even if the user types a number. Why might this cause problems in a program, and how could you fix it?
2. What's the difference between `Number()`, `parseInt()`, and `parseFloat()`? Can your group think of a situation where you'd want one over the others?
3. If you called `.toUpperCase()` on a variable, does it change the original variable, or does it just give you a new value? Discuss and test your prediction.
4. Why might a program need to check the *length* of a string a user typed in?
5. What real-world scenario can you think of where converting string to a number (or a number to string) would be essential?

Part 2: Predict the Output

Snippet A

```
let input = "42";
let converted = Number(input);
console.log(input + 8);
console.log(converted + 8);
```

Snippet B

```
let name = "  Chidinma  ";
console.log(name.trim());
console.log(name.length);
```

Snippet C

```
let word = "javascript";
console.log(word.toUpperCase());
console.log(word);
```

Part 3: Debugging Challenge

```
// 1

let userAge = prompt("Enter your age:");
let nextYearAge = userAge + 1;
console.log("Next year you will be " + nextyearAge);

// 2

let city = "lagos"
console.log(city.ToUpperCase());
```

Discuss what's wrong before fixing it.

Part 4: Collaborative Technical Challenge

Scenario: Build a "Username Generator" that asks for a person's first name and favorite number, converts the number properly, and creates a username by combining the lowercase first name with the number.

Step 1 — Understand the Problem: Explain what the finished program should do.

Step 2 — Identify Inputs: What does the program need from the user?

Step 3 — Identify Processing: What conversions and string methods are needed?

Step 4 — Identify Outputs: What should the final username look like?

Step 5 — Plan the Solution: Write your plan in plain English first.

Step 6 — Write the Code: Use `.toLowerCase()`, type conversion, and template literals or concatenation.

Step 7 — Test the Program:

- Name: "SARAH", Number: 7
- Name: "kunle", Number: "12" (typed as text)
- Name: " Ada " (with extra spaces), Number: 0

- Name: "Femi", Number: 3.5

Step 8 — Improve the Solution: Should you `.trim()` the name in case someone types extra spaces? Discuss why or why not.

Part 5: Extension Challenge

Add a check using `.length` — if the name is longer than 8 characters, only use the first 8 characters in the username.

Activity 4: Text and Numbers

Topics Covered

String Methods, Math Methods

Learning Objective

Students should be able to select the right built-in method for a task, whether working with text or numbers, and explain what each method actually does.

Part 1: Group Discussion

1. Why might a program need `Math.random()`? What kinds of programs use randomness?
2. If you wanted to check whether a piece of text contains a certain word (like checking if an email contains "@"), what string method might help?
3. Compare `.slice()` and `.split()`. What does each one actually produce?

Part 2: Predict the Output

Snippet A

```
console.log(Math.round(4.5));  
console.log(Math.floor(4.9));  
console.log(Math.ceil(4.1));
```

Snippet B

```
let phrase = "I love JavaScript";  
console.log(phrase.includes("love"));  
console.log(phrase.split(" "));
```

Predict the output of the code snippet.

Snippet C

```
let num = 7;  
console.log(Math.max(num, 10, 3));  
console.log(Math.min(num, 10, 3));
```

Part 3: Debugging Challenge

```
let score = 87.6;
let rounded = Math.Round(score);
console.log("Rounded score: " rounded);

let sentence = "Learning to code is fun"
let firstWord = sentence.split(" ")[0]
console.log(firstword);
```

Discuss the errors as a group before fixing them.

Part 4: Collaborative Technical Challenge

Scenario: Build a "Grade Rounder and Reporter" that asks for a student's exact numeric score, rounds it, and builds a message reporting the rounded score along with the student's name in all uppercase.

Step 1 — Understand the Problem: Explain the goal in your own words.

Step 2 — Identify Inputs: What does the program need from the user?

Step 3 — Identify Processing: What Math and string methods will you need?

Step 4 — Identify Outputs: What should the final message look like?

Step 5 — Plan the Solution: Write your plan in plain English first.

Step 6 — Write the Code: Combine Math methods, string methods, and template literals.

Step 7 — Test the Program:

- Name: "grace", Score: 89.5
- Name: "Chuka", Score: 100
- Name: " Bisi " (extra spaces), Score 59.3

Step 8 — Improve the Solution: Discuss whether trimming the name before uppercasing it would make the program more reliable.

Part 5: Extension Challenge

Add `Math.random()` to simulate a "bonus point" between 0 and 5 that gets added to the score before rounding. Discuss how you'd make sure the random bonus is always a whole number.

Activity 5: Decision Makers

Topics Covered

Control Flow, Conditional Statements

Learning Objective

Students should be able to trace how a program decides between different paths, write conditions using comparison and logical operators, and explain their reasoning aloud.

Part 1: Group Discussion

1. What does it mean for a program to make a "decision"? Give a real-life example of a decision that could be written as an `if` statement.
2. What's the difference between `=`, `==`, and `===`? Why might using the wrong one cause bugs?
3. If you have an `if`, an `else if`, and an `else`, can more than one block run? Discuss and explain your reasoning.
4. How do `&&` (AND) and `||` (OR) change the way a condition behaves? Can your group come up with an everyday sentence that uses "and" versus "or" logically?

Part 2: Predict the Output

Snippet A

```
let temperature = 15;
if (temperature > 30) {
  console.log("It's hot!");
} else if (temperature > 15) {
  console.log("It's warm.");
} else {
  console.log("It's cool.");
}
```

Snippet B

```
let age = 20;
let hasID = false;
if (age >= 18 && hasID) {
  console.log("You may enter.");
} else {
  console.log("Entry denied.");
}
```

Snippet C

```
let password = "1234";
if (password === "0000") {
  console.log("Password changed!");
} else {
  console.log("No change.");
}
```

Part 3: Debugging Challenge

let hour = 14;

```
if (hour < 12) {  
  console.log("Good morning!");  
else if (hour < 18) {  
  console.log("Good afternoon!");  
} else {  
  console.log("Good evening!")  
}
```

Discuss what's wrong before fixing it.

Part 4: Collaborative Technical Challenge

Scenario: Build a "Movie Ticket Pricer" that asks for a person's age and whether it's a weekday or weekend, then decides the ticket price using conditional logic (e.g., discounts for children, seniors, or weekday showings).

Step 1 — Understand the Problem: Explain the pricing rules your group wants to use, in plain English.

Step 2 — Identify Inputs: What information does the program need?

Step 3 — Identify Processing: What conditions determine the final price?

Step 4 — Identify Outputs: What should the program display?

Step 5 — Plan the Solution: Write out your decision logic as plain-English rules before coding.

Step 6 — Write the Code: Implement your rules using `if/else if/else`.

Step 7 — Test the Program:

- Age: 8, Day: weekday

- Age: 70, Day: weekend
- Age: 30, Day: weekend
- Age: 17, Day: weekday

Step 8 — Improve the Solution: Are there overlapping conditions that could cause confusion? Discuss how to order your conditions to avoid mistakes.

Activity 6: Shortcuts and Choices

Topics Covered

Ternary Operators (with review of Conditional Statements)

Learning Objective

Students should be able to rewrite simple if/else logic as a ternary expression, and judge when a ternary makes code clearer versus when it makes code harder to read.

Part 1: Group Discussion

1. A ternary operator is often described as a "shortcut" for if/else. What do you think that means, based on what you've seen?

Look at this if/else block. As a group, try to say out loud (without writing code yet) what the equivalent ternary might look like:

```
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

2. Can a ternary operator replace an `if/else if/else` with three branches? Why or why not?
3. When might a ternary operator make code *harder* to read instead of easier? Discuss a case where you'd stick with regular `if/else`.

Part 2: Predict the Output

Snippet A

```
let score = 72;  
let result = score >= 50 ? "Pass" : "Fail";  
console.log(result);
```

Snippet B

```
let cartTotal = 0;  
let message = cartTotal > 0 ? "Proceed to checkout" : "Your cart is empty";  
console.log(message);
```

Snippet C

```
let stock = 5;  
console.log(`Stock status: ${stock > 0 ? "Available" : "Out of stock"}`);
```

Part 3: Debugging Challenge

```
let temperature = 28;
let feeling = temperature > 25 ? "hot" : "cold";
console.log(feeling)

let isMember = true
let discount = isMember ? 10 : 0;
console.log(`Discount: ${discount}%`);
```

Discuss the errors before fixing them.

Activity 7: Repeat After Me

Topics Covered

Loops

Learning Objective

Students should be able to trace how a loop repeats, identify when a loop will run forever by mistake, and choose appropriate loop conditions for a task.

Part 1: Group Discussion

1. Why would a programmer use a loop instead of just writing the same line of code multiple times?
2. What's the difference between a **for** loop and a **while** loop? When might you prefer one over the other?
3. What is an "infinite loop," and what usually causes one by accident? Discuss what you'd look for when debugging one.
4. If a loop is supposed to run 5 times but only runs 4, what part of the loop would you check first?
5. Think of something you do repeatedly in real life (like brushing your teeth in circular motions). How would you describe that action using loop logic (a starting point, a condition, and a step)?

Part 2: Predict the Output

Snippet A

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Snippet B

```
let count = 3;  
while (count > 0) {  
  console.log(`Countdown: ${count}`);  
  count--;  
}  
console.log("Liftoff!");
```

Snippet C

```
for (let i = 0; i < 10; i = i + 2) {  
  console.log(i);  
}
```

Part 3: Debugging Challenge

```
// 1

let total = 0;

for (let i = 1; i <= 5; i++) {
  total = total + i
}

console.log("Total: " + total);


// 2

let count = 5;
while (count > 0) {
  console.log(count);
}
```

Discuss the errors before fixing them.

Activity 8: Reusable Machines

Topics Covered

- What Are Functions?
- Function Declarations
- Functions with Parameters

Learning Objective

Students should be able to explain why functions help avoid repeated code, write a function with parameters, and trace how arguments get passed in when a function is called.

Part 1: Group Discussion

1. Imagine a vending machine: you put in money and a selection, and it gives you a snack. How is that similar to how a function works?
2. What's the difference between a "parameter" and an "argument"? Many beginners mix these up — discuss until your group agrees on clear definitions.
3. Why would a programmer create a function instead of just writing the same block of code every time it's needed?
4. If a function is declared but never called, what happens when the program runs? Discuss your prediction.
5. Can a function have more than one parameter? What do you think happens if you call the function without providing all the arguments it expects?

Part 2: Predict the Output

Snippet A

```
function greet(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greet("Ifeoma");  
greet("David");
```

Snippet B

```
function addNumbers(a, b) {  
  console.log(a + b);  
}  
  
addNumbers(4, 7);  
addNumbers(10);
```

Snippet C

```
function sayHi() {  
  console.log("Hi there!");  
}  
  
console.log("Before calling function");  
sayHi();  
console.log("After calling function");
```

Part 3: Debugging Challenge

```
function calculateArea(length, width) {  
  console.log(length * width)  
  
function displayMessage(msg) {  
  console.log(msg);  
}  
  
displayMessage("Area calculator ready");  
CalculateArea(5, 10);
```

Discuss the errors before fixing them.

Part 4: Collaborative Technical Challenge

Scenario: Build a "Tip Calculator Toolkit" made of two functions: one that calculates a tip based on a bill amount and tip percentage, and another that displays a formatted final message.

Step 1 — Understand the Problem: Explain what each function should do, separately.

Step 2 — Identify Inputs: What parameters does each function need?

Step 3 — Identify Processing: What calculation happens inside the tip function?

Step 4 — Identify Outputs: What should each function produce or display?

Step 5 — Plan the Solution: Decide as a group how many functions you'll use and what each one is responsible for, before writing code.

Step 6 — Write the Code: Declare your functions with parameters, then call them with test values.

Step 7 — Test the Program

Part 5: Extension Challenge

Add a third function that calculates the total bill split evenly between a given number of people

Activity 9: Functions Expressions and Arrow Functions

Topics Covered

- Anonymous Functions
- Arrow Functions (with review of Function Declarations and Parameters)

Learning Objective

Students should be able to recognize the same logic written as a function declaration, an anonymous function, and an arrow function, and explain what's different (and what's the same) between them.

Part 1: Group Discussion

1. What makes a function "anonymous"? If it has no name, how does JavaScript know when to run it?

Look at these three versions of the same idea. As a group, discuss what's different about how each is written, even though they do the same thing:

```
function square(num) {  
  return num * num;  
}  
  
const square2 = function(num) {  
  return num * num;  
};  
  
const square3 = (num) => {  
  return num * num;  
};
```

2. Why might a programmer choose an arrow function over a regular function declaration? Can your group think of a downside to always using arrow functions?
3. If you stored an anonymous function in a variable, how would you call it later?
4. Arrow functions are sometimes written even shorter, like `(num) => num * num`. What do you think happened to the curly braces and the word "return"?

Part 2: Predict the Output

Snippet A

```
const multiply = function(a, b) {  
  return a * b;  
};  
  
console.log(multiply(3, 4));
```

Snippet B

```
const double = num => num * 2;

console.log(double(6));
console.log(double(0));
```

Snippet C

```
const introduce = (name, age) => {
  console.log(`My name is ${name} and I am ${age} years old.`);
};

introduce("Zainab", 22);
```

Part 3: Debugging Challenge

```
// 1
const greetUser = (name) => {
  console.log("Welcome, " + name)
}

greetuser("Emeka");

// 2
const subtract = (a, b) => a - b
console.log(subtract(10, 4))
```

Discuss the errors before fixing them.

Part 4: Collaborative Technical Challenge

Scenario: Build a small "Math Helper Toolkit" using arrow functions for common calculations a student might need: converting a percentage to a decimal, calculating a rectangle's area, and converting Celsius to Fahrenheit.

Step 1 — Understand the Problem: Explain what each small helper function needs to do.

Step 2 — Identify Inputs: What parameters does each arrow function need?

Step 3 — Identify Processing: What's the formula or calculation inside each one?

Step 4 — Identify Outputs: What should each function return, and how will you display the results?

Step 5 — Plan the Solution: Decide together how many arrow functions you need and what each is responsible for.

Step 6 — Write the Code: Write each helper as an arrow function, then call each one with test values and log the results.

Step 7 — Test the Program:

- Percentage to decimal: 25
- Rectangle area: length 4, width 9
- Celsius to Fahrenheit: 35
- Celsius to Fahrenheit: 100
- Rectangle area: length 10, width 5

Step 8 — Improve the Solution: Discuss whether writing these as arrow functions felt more natural than regular function declarations, and why.

Part 5: Extension Challenge

Add an arrow function that takes a Fahrenheit temperature and converts it back to Celsius, then use it to check that your two conversion functions "undo" each other correctly.

Activity 10: Build-a-Program Challenge

Topics Covered

- Variables
- Data Types
- Operators
- Template Literals
- Type Conversion
- String/Math Methods
- Conditionals
- Ternary Operators
- Loops
- Functions (including Arrow Functions)

Learning Objective

Students should be able to combine everything they've learned into one cohesive program, making deliberate design decisions and explaining their reasoning as a group.

Part 1: Group Discussion

1. Looking back at everything you've learned this course, which concept do you think connects to the most other concepts? Why?
2. If you were explaining this entire course to a friend who has never coded, in two or three sentences, what would you say JavaScript actually lets you *do*?
3. When building a larger program, why might it help to break it into small functions instead of writing one giant block of code?
4. Think about a bug you fixed earlier in this course. What did that experience teach you about how to approach debugging in general?
5. As a group, discuss: which combination of concepts (e.g., loops + functions, or conditionals + ternaries) do you feel most confident about, and which still feels shaky?

Part 2: Predict the Output

Snippet A

```
const getDiscount = (total) => total >= 100 ? total * 0.1 : 0;

let orderTotal = 120;
let discount = getDiscount(orderTotal);
console.log(`Discount: ${discount}`);
console.log(`Final total: ${orderTotal - discount}`);
```

Snippet B

```
function classifyNumbers(limit) {
  for (let i = 1; i <= limit; i++) {
    console.log(i % 2 === 0 ? `${i} is even` : `${i} is odd`);
  }
}

classifyNumbers(4);
```

Snippet C

```
const formatName = (name) => name.trim().toUpperCase();

let rawInput = "  kelechi  ";
console.log(`Welcome, ${formatName(rawInput)}!`);
```

Part 4: Collaborative Technical Challenge

Scenario: Build a complete "Order Checkout Program" that combines everything from this course:

- Collects an item name, price, and quantity using `prompt()`
- Converts inputs to the correct data types
- Uses a function to calculate the subtotal
- Uses a conditional (or ternary) to apply free shipping if the total is above a certain amount
- Displays a final formatted receipt using a template literal

Step 1 — Understand the Problem: As a group, describe the entire program from start to finish in your own words.

Step 2 — Identify Inputs: List every piece of information the program needs from the user.

Step 3 — Identify Processing: List every calculation, conversion, and decision the program must make, in order.

Step 4 — Identify Outputs: Describe exactly what the final receipt should look like.

Step 5 — Plan the Solution: Write the full plain-English algorithm before writing any code. Assign no fixed roles — just talk through the logic together until everyone agrees on the plan.

Step 6 — Write the Code: Build the program using functions, conditionals/ternaries, and template literals together.

Step 7 — Test the Program:

- Item: "Backpack", Price: 30, Quantity: 2
- Item: "Pen", Price: 1.5, Quantity: 20

Step 8 — Improve the Solution: As a group, review your finished program together. Is there any repeated logic that could be turned into its own function? Are your variable and function names clear to someone reading it for the first time?

Submission Requirements

1. Create a JavaScript (.js) file and an HTML (.html) file, link them together, and test all your code properly.
2. Use Git and GitHub to manage and submit your work.
3. Group leaders should submit the completed work on behalf of all group members.
4. Ensure that the GitHub repository link is included in the submission form.
5. Group leaders should submit their group's work using [the submission form here](#).